



Program: BS Computer Science

Semester: Fall 2025

Credit Hour(s): 1

Pre-requisite(s): EE1005 Digital Logic Design

Lab: Computer Organization and Assembly Language

Instructor: Engr. Muhammad Qasim – muhammad.qasim@nu.edu.pk

Lab # 09: Bitwise Logical Operations and Masking, Subroutines, CALL and RET

Lab Objective

To understand and implement **Bitwise Logical Operations** and **Masking Techniques** for low-level data manipulation, and to implement **Subroutines** using the CALL and RET instructions to enable code reusability and modular programming.

Part I: Bitwise Logical Operations and Masking

4.5 Theoretical Background: Bitwise Logical Operations

The 8088 processor provides a set of logical operations that function at the **bit level**. These operations are performed in parallel on the corresponding bits of the operands (byte or word).

Instruction	Operation	Description
AND	Logical Bitwise AND	A bit in the result is set only if both corresponding bits in the operands are set
OR	Logical Bitwise Inclusive OR	A bit in the result is set if either or both corresponding bits in the operands are set
XOR	Logical Bitwise Exclusive OR	A bit in the result is set if the corresponding bits in the operands are different
NOT	Logical Bitwise NOT	Inverts every bit of the single operand.

For example, the instruction AND AX, BX results in 16 parallel AND operations, one for each bit position from 0 to 15³.

4.6 Theoretical Background: Masking Operations

Masking is the technique of using a bitwise logical operation with a specific value (the **mask**) to selectively manipulate the bits of an operand without affecting the other bits⁴.



Operation	Logical Instruction Used	Mask Value Logic
Selective Bit Clearing	AND	Use 0s at positions to be cleared and 1s at positions to be retained[cite: 5689, 5690]. (<i>Example: To clear the lower nibble, use a mask of F0h</i>)[cite: 5691].
Selective Bit Setting	OR	Use 1s at positions to be set and 0s at positions to be retained[cite: 5693, 5694]. (<i>Example: To set the lower nibble, use a mask of 0Fh</i>)[cite: 5695].
Selective Bit Inversion	XOR	Use 1s at positions to be inverted and 0s at positions to be retained[cite: 5717, 5718].

Lab Procedure: Selective Bit Testing

The TEST instruction is the most common and powerful way to test specific bits.

- **Instruction:** TEST destination, source
- **Functionality:** TEST performs a logical **AND** operation between its two operands.
- **Key Feature:** It updates the flags register (Zero Flag, Sign Flag, etc.) based on the result but **does not modify either the destination or source operands**. This makes it a non-destructive test.
- **Procedure:**
 1. Create a **mask** with a **1** in the bit position you want to check and **0s** everywhere else.
 2. Execute the TEST instruction: TEST operand, mask.
 3. Check the **Zero Flag (ZF)** using a conditional jump:
 - If the result of the logical AND was **zero** (i.e., the desired bit was **cleared/0**), the Zero Flag will be set (ZF=1). Use JZ (Jump if Zero).
 - If the result of the logical AND was **non-zero** (i.e., the desired bit was **set/1**), the Zero Flag will be cleared (ZF=0). Use JNZ (Jump if Not Zero).

Part II: Subroutines, CALL and RET

5.1 Theoretical Background: CALL and RET



When a block of code (like a sorting or multiplication algorithm) needs to be used multiple times, it is inefficient to copy the code everywhere⁹⁹⁹. This problem is solved by **subroutines** (or procedures) using the CALL and RET instructions.

Instruction	Function	Mechanism
CALL	Temporary Diversion	Transfers program execution control to the start of the subroutine[cite: 5835, 5854]. It automatically pushes the return address (the address of the instruction following the CALL) onto the Stack [cite: 5970, 6063, 10897].
RET	Return	Transfers program execution control back to the calling instruction[cite: 5876, 6062]. It automatically pops the return address from the Stack into the Instruction Pointer (IP) and increments the Stack Pointer (SP)[cite: 6063, 10897].

The **Stack** provides the automatic coordination mechanism, ensuring that the processor knows precisely where to return after the subroutine completes¹⁰¹⁰¹⁰. This provides a controlled **round-trip** program flow¹¹.

5.2 Our First Subroutine: Bubble Sort Example

The conversion of a frequently used algorithm, such as **Bubble Sort**, into a subroutine is a classic example of achieving code reusability¹².

Parameters

To make the subroutine flexible, it must be able to operate on different data each time it's called¹³. **Parameters** are pieces of information that change from one invocation to the next, such as the array's starting address and the number of elements¹⁴.

In this example, parameters are passed via registers¹⁵¹⁵¹⁵¹⁵:

- **BX:** Holds the start address of the array (Base/Address parameter)¹⁶¹⁶¹⁶¹⁶.
- **CX:** Holds the count of elements (Count parameter)¹⁷¹⁷¹⁷¹⁷.

Example 5.2: Calling a Subroutine Twice

The power of subroutines is demonstrated by calling the same bubblesort routine for two different arrays of different sizes without rewriting the sorting code¹⁸.

Conceptual Code Flow:

; --- Main Program (Caller) ---

start:

; First Call



mov bx, data ; Send start address of array 'data'

mov cx, 10 ; Send count of elements (10)

call bubblesort ; Execute subroutine 1

; Second Call

mov bx, data2 ; Change parameters: Send start address of array 'data2'

mov cx, 20 ; Change parameters: Send count of elements (20)

call bubblesort ; Execute subroutine 2 (same code, different data)

mov ax, 0x4c00 ; Terminate program

int 0x21

; --- Subroutine ---

bubblesort:

; ... Sorting logic ...

ret ; Return to instruction following the CALL

The first CALL returns to the instruction mov bx, data2. The second CALL returns to the instruction mov ax, 0x4c00. This flow is essential for structured and modular assembly programming.

Lab Tasks: (should be checked with in the lab)

Task 1: Using masking generate a pyramid in m1 memory of debugger as shown below:

FF FF FF FF FF FF FF

FF FF 00 FF FF FF FF

FF 00 00 00 FF FF FF

00 00 00 00 00 FF FF

Task 2: Write assembly code to find a factorial of a number using subroutine